SENSIBle: A Highly Scalable SENsor DeSIgn for Path-Based Age Monitoring in FPGAs

Zana Ghaderi, *Student Member, IEEE*,
Mohammad Ebrahimi,
Zainalabedin Navabi, *Senior Member, IEEE*,
Eli Bozorgzadeh, and Nader Bagherzadeh, *Fellow, IEEE*

Abstract—This paper proposes a highly scalable sensor design for late transition detection in FPGA based platforms. Transition delays occur because of aging mechanisms such as *Biased Temperature Instability* (BTI) and *Hot Carrier Injection* (HCI). We propose a sensor clock (SCLK) that is a function of minimum slack time of a set of paths selected for age monitoring. There will be one such clock for many sensors as are needed in an entire FPGA. Our proposed sensor architecture makes it possible for a single SCLK to be shared by all sensors. Additionally, the proposed sensor occupies one slice (basic FPGA logic block), which leads to low area, power, and performance overhead. Using Artix-7-based board, experimental results demonstrate that the proposed aging sensor detects aging earlier than existing sensors and provides less power and performance overheads.

Index Terms—Aging, FPGA, late transition, scalability, sensor

1 Introduction

FPGAs are broadly deployed to accelerate computation in various applications ranging from embedded applications such as automotive, vision, and medical devices to datacenter applications. Fabricated in latest advanced silicon technology and deployed for highly computationally intensive kernels, FPGAs face reliability challenges such as aging [1], [2], [3]. BTI and HCI aging mechanisms induce delay degradation to FPGA resources and increase leakage power consumption [4], [5], [6], [7], [8]. The delay degradation in logic and routing resources along *Critical Paths* (CPs) of a design on FPGA results in late transitions that can cause timing failures.

Delay degradation (aging rate) in BTI and HCI mechanisms is exponential function of temperature and non-linear function of stress time. Each resource in FPGA can be possibly in a critical path of target applications with different temperature and *Stress Rate* (SR) maps. This means different CPs experience different temperatures and stresses, thus different aging rates. Therefore, delay of CPs with longer *slack times* may exceed smaller ones (including longest critical path) [8]. This paper focuses on on-chip aging sensors for critical path delay monitoring in FPGAs.

On-chip aging sensors that are referred to as application dependent sensor [5], [6], [9], are deployed along the paths in a circuit to detect any increase in delay of the paths for system level aging mitigation techniques [9], [10]. A path has a *slack time* relative to the longest critical path or operating clock period (MCLK). Any transition in *slack time* is an indication of aging that needs to be detected by aging sensors. While critical paths with negligible *slack times* are more vulnerable to timing failure due to aging, it is shown that some of near-critical paths with relatively small non-zero *slack time* often have higher aging rates [8]. Hence, by selecting and monitoring such "near-critical" paths as *Representative Critical Paths* (RCPs), we can potentially detect aging proactively to avoid functional failures caused by timing failures in the entire system [8]. Such RCPs even tend to exceed the longest CP's delay due to aging.

Inserting aging sensors on such RCPs' endpoint avoids performance loss due to added delay of sensors to RCPs. To find these RCPs we deployed and improved our RCP selection algorithm in [8]. This algorithm finds RCPs that have higher aging rates than CPs with smaller *slack time*. However, if aging rates of CPs with smaller *slack times* (including longest CP) are high enough that may result to timing failure before detection of aging on other selected RCPs the algorithm selects them for monitoring. Hence, negligible performance loss is unavoidable.

Slack-based aging sensor architecture and design for such RCPs is challenged by various factors. Due to a large number of RCPs in a design, the number of required sensors will be high [8]. Each aging sensor requires a sensor clock (SCLK) and state-of-the-art aging sensors cannot share a single clock source [5], [6]. Given that the number of clock generation modules is limited on an FPGA, existing aging sensor designs are not scalable to be deployed in a large scale for RCPs (Fig. 1a). This becomes more important in multiple clock domain designs. Another challenge is sensor placement at the RCPs' endpoints. Sensors with higher resource utilization (logic and clock resources) incur more stringent placement constraints. If aging sensors utilize fewer logic resources and share a clock resource, they are more likely to be inserted near RCPs' endpoints. This leads to higher scalability and more accurate operation of sensors with less overhead. Higher accuracy (or sensitivity) leads to sooner aging detection than previous works [5], [6]. This helps system level aging mitigation methods to react in proper times by having more precise aging information in RCPs.

This paper presents a highly scalable sensor architecture architecture by utilizing the minimum *slack time* of selected RCPs to build a shared sensor clock (SCLK) named as SENSIBLE. The proposed multi-sensor clock design has lower area and power overhead in comparison with state-of-the-art aging sensors. As illustrated in Fig. 1b, in SENSIBLE we can use the same clock generator for a group of sensors. Additionally, the proposed sensor only occupies one basic FPGA logic block (e.g., a slice in Xilinx Artix FPGAs).

Programmable routing resources between programmable logic blocks (CLBs) on FPGAs impose significant delay overhead in a sensor design, which may sometimes exceed the aging-induced delay degradation. By containing the sensor in one logic block and avoiding programmable routing resources, our proposed aging sensors are able to detect aging earlier (i.e., higher accuracy) than existing aging sensors, and yet, they have lower area and power overheads. Earlier (more accurate) aging detection before happening of timing failure helps the system level aging mitigation techniques to react sooner properly. For example, by changing the placement (configuration bits) more aged regions we are able to mitigate aging in reconfigurable architectures [11].

The experimental results on Artix7-based board show that the SENSIBLE is a scalable low-overhead aging sensor and hence, it can be inserted in designs on FPGAs in large scale with negligible impact on design performance. Our experimental results support this claim that unlike previous works [5], [6], our proposed sensor design with lower logic resource utilization can fit as closely as possible to path endpoint CLB so as to avoid programmable routing resources. Due to higher accuracy, the aging along CPs are detected earlier than using aging sensors in [5], [6]. To the best of

- *Z. Ghaderi, E. Bozorgzadeh, and N. Bagherzadeh are with the Computer Science Department of UC Irvine, Irvine, CA 92697. E-mail: {zghaderi, eli, nader}@uci.edu.*
- *M. Ebrahimi and Z. Navabi are with the Electrical and Computer Engineering Department, University of Tehran, Tehran 14155-6619, Iran. E-mail: {ebrahimi.mo, navabi}@ut.ac.ir.*

Manuscript received 27 May 2016; revised 5 Oct. 2016; accepted 26 Oct. 2016. Date of publication 27 Oct. 2016; date of current version 13 Apr. 2017.
Recommended for acceptance by J.D. Bruguera.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TC.2016.2622688

0018-9340 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.

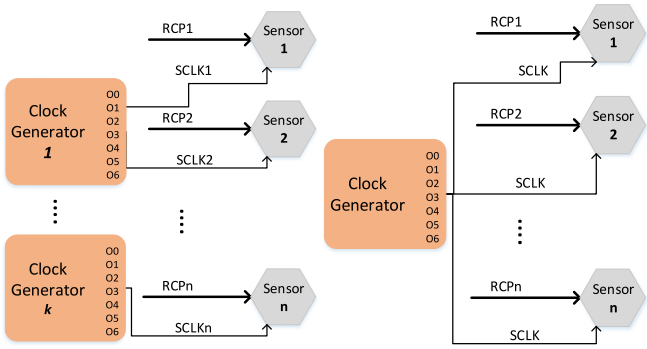


Fig. 1. Multiple aging sensor architecture in FPGA.

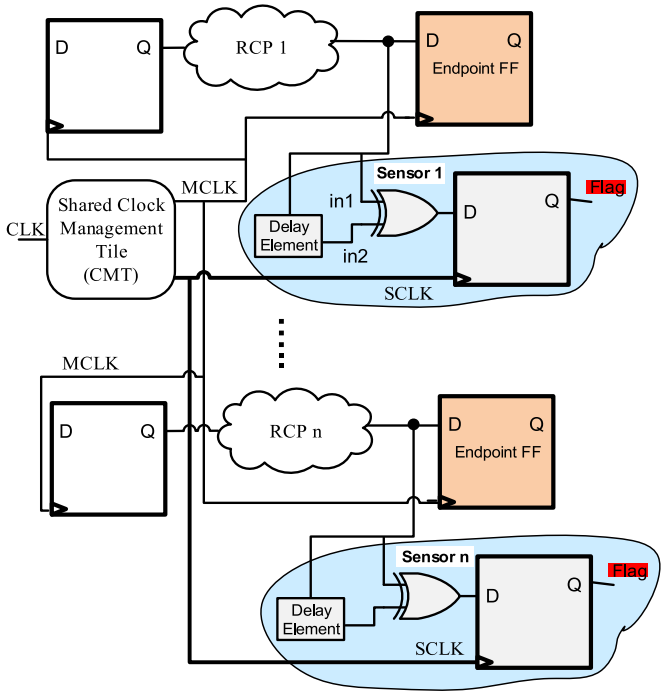
our knowledge this is the first attempt for scalable design of aging sensors on FPGAs which can be applied for ASIC design as well.

2 Related Works

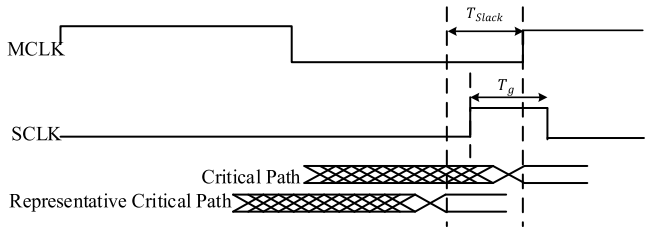
Logic-based aging sensors for ASIC designs have been proposed in the literature. For instance, sensors presented in [9], [12] measure delay degradation in a circuit due to transistor aging. In [9], two approaches for detection and correction of late transition due to NBTI are proposed, which either impose performance or area overhead to the circuit. The sensor introduced in [12] is based on two ring oscillators, one as a reference, and the other one under stress conditions. The difference between their frequencies is a representation of their delay difference. In [13], a ring oscillator based multi-purpose sensor on FPGA is presented. Although their method is suitable to extract the FPGA delay, they cannot be used for monitoring aging on CPs (i.e., not application dependent).

Application dependent aging sensors (by monitoring aging along CPs) for FPGA-based designs are proposed in [5], [6]. In order to detect transitions on path endpoints near the active clock edge, they monitor RCPs. In [6], an FPGA-based aging sensor is proposed using more than one slice. A similar approach is used in [5], with a difference that their proposed sensor is able to adjust the observation interval dynamically. Since such aging sensors require different clocks, multiple clock sources are required (See more details in Section 5). This not only reduces their scalability due to limited numbers of clock generation modules, but also increases the area and power overheads. Additionally, the aging sensors occupy more than a basic logic block (e.g., slice) and hence, they are forced to connect to the RCPs' endpoint through programmable routing resources (outside the slice), with a higher delay. In addition, due to higher resource utilization (clock resources and logic), it is less likely to be able to place the sensors close to the RCPs' endpoints. Detailed comparison in Section 5 is provided.

Some of the related works try to mitigate aging in reconfigurable architecture at different levels of granularity by changing the placement of design inside an FPGA [7], [11], [14]. [7], [11] proposed offline aging mitigation techniques by profiling the aging rates of different blocks in an application to increase chip lifetime or avoid timing failure at runtime. However, the applications' behavior (temperature and stress) at runtime might be different and result to impaired solutions which leads to online aging mitigation techniques [14]. Aging sensors on RCPs will be one way of online monitoring. Therefore, sensors with lower overheads and higher sensitivity is a need. Lower overheads (area, power, performance, number of required clock generators, clock routing) leads to higher scalability and lower design complexity. In this work, we amended our proposed RCP selection method in [8] to find required RCPs for aging monitoring in FPGAs (Section 5).



(a) Block diagram of the proposed aging sensor



(b) MCLK and SCLK representation

Fig. 2. Proposed sensor architecture and design.

3 Proposed Aging Sensor (SENSIBLE)

Operation of SENSIBLE is based on the fact that several paths (selected RCPs) in a circuit have non-zero *slack times* that are relatively close to the *slack time* of the critical paths. A transition on such path's endpoints during the *slack time* is known as an erroneous behavior. Therefore, detection of any transition on a path's endpoint during the *slack time* caused by aging mechanism, is considered as an aging effect.

The design of the proposed method is shown in Fig. 2a. The aging sensor is potentially placed on a near-critical path's (RCP1) endpoint. The sensor consists of a FF, an XOR gate, and a *Delay Element* (DE). For clocking the sensors, a sensor clock (SCLK) is generated that is synchronized with the main system clock (MCLK) using *Clock Management Tiles* (CMTs). Fig. 2b shows the timing diagram of SCLK and MCLK. The SCLK frequency is the same as that of MCLK. SCLK makes a 0-to-1 (active edge) transition during the *slack time* before the active edge of MCLK.

Today's FPGAs have a feature for generating different clock signals using embedded standard resources called *Clock Management Tiles* (CMTs), which consist of *Mixed-Mode Clock Manager* (MMCM) and/or PLLs. For a given clock period T_{clk}, CMTs allow a configurable phase shift to take any discrete value; $phase\_shift = N \times T_{clk}/256$, where N is an integer in the range $-255 < N < 255$. In addition, CMTs allow generation of various duty cycles for a given clock period. Therefore, we can generate a shifted clock signal with a desired duty cycle for the sensor, as required by the proposed aging sensor in this work.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.

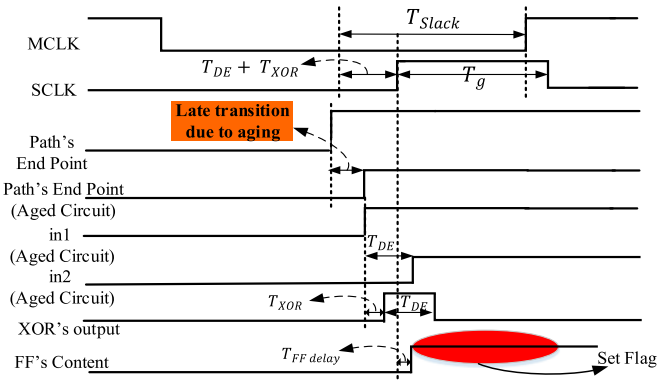


Fig. 3. The sensor operation in the presence of late transition happened due to aging.

Fig. 3 shows the timing diagram of the sensor presented in Fig. 2a (for RCP1). As shown, a late transition generates a positive pulse with pulse width of TDE at the output of the XOR gate. The sensor's FF is initialized to '0'. The XOR gate receives data from the near CP (in1) (RCP1), and its time-shifted data (in2) by delay element. Because of the delay element, any transition at the output of the RCP driving this sensor will cause a positive pulse on the XOR output. If this pulse happens outside of the *slack time*, it will safely pass without affecting the FF value. On the other hand, due to the induced delay by aging in the RCP, the aging sensor's FF will capture the pulse at XOR output during *slack time* window. This occurs on the positive edge of SCLK where FF setup and hold times are also considered.

This positive pulse causes a '1' at the FF output. When there is no aging effect, the path's endpoint does not have any transition that can be captured by SCLK during *slack time*. Therefore, there will be no difference between the XOR inputs. XOR output remains at '0' and hence, the FF remains at '0' indicating no late transition.

In order to ensure correct operation of the sensor, T_{Slack} must be chosen based on *slack times* of selected paths sharing the same clock. Given a set of RCP *i* with *slack time* S_i, T_{Slack} is the minimum of all *slack times*.

$$T_{Slack} = \min(S_1, S_2, \ldots, S_n) \tag{1}$$

This condition guarantees that none of the selected paths has any transition in T_{Slack}. Valid changes at *in1* input of XOR gate can occur up to the start of T_{Slack}. Since *in2* is formed by delaying *in1* by T_{DE}, we can expect *in2* to change T_{DE} after the start of T_{Slack}. Considering T_{XOR} gate delay, the clocking of the FF must be after $T_{DE} + T_{XOR}$ in order to capture *slack time* violations. It should be noted that as long as the aforementioned condition for SCLK start point is met, its duty cycle could exceed the duty cycle of MCLK. This leads to the easier design of SCLK for our proposed sensor.

To determine the delay of the *delay element* (T_{DE}), the timing difference between the XOR gate inputs must be long enough such that the XOR gate can propagate this difference to its output (e.g., generating positive pulse at its output). Hence, the first condition to be satisfied by T_{DE} is:

$$T_{DE} \geq T_{XOR} \tag{2}$$

This pulse propagates to the output of XOR gate (input of FF). Also, T_{DE} must be long enough so that FF setup and hold times are not violated, i.e.,

$$T_{DE} \geq T_{FF\_setup\_hold\_time} \tag{3}$$

From (4) and (5), we can conclude:

$$T_{DE} \geq Max(T_{XOR}, T_{FF\_setup\_hold\_time}) \tag{4}$$

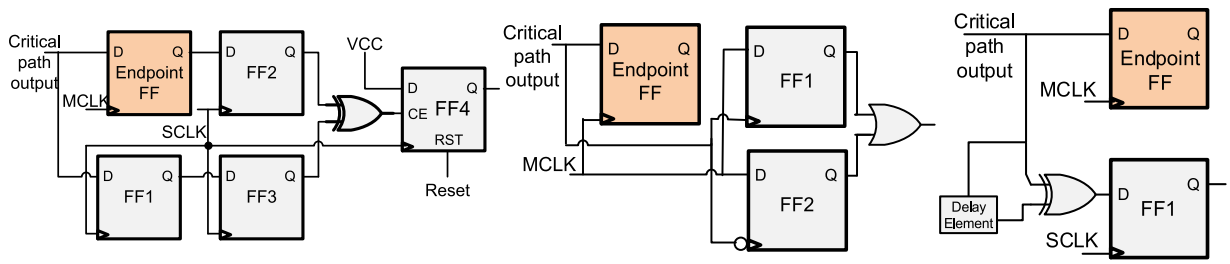
The implementation of the proposed sensor on FPGA resources is shown in Fig. 5c. In Xilinx Artix (or Virtex) FPGAs, our sensor occupies only one slice. Recall that *slice* is the basic logic block in Xilinx FPGA devices. Therefore, our proposed sensor only uses the intra-slice interconnect resources for connectivity and does not require programmable routing resources. Hence, the proposed sensor has higher accuracy in comparison to available aging sensors in the literature. Moreover, *delay element* (DE) is implemented using two LUTs connected serially.

When there is no unused slice inside the endpoint CLB, an unused slice in the nearby CLBs will be selected as the sensor slice. In this case, the programmable routing resources are used. To avoid such scenarios, the placement tools can reserve a slice in the endpoint CLB for sensor insertion. Unlike previous works that propose aging sensors with two slices [5], [6], our proposed sensor only occupies one slice and hence, the probability of finding an unused slice inside the endpoint CLB without changing original design placement is higher. We use a greedy local search to find the closest unused slices (more details in Section 6.2).

Since aging is a gradual mechanism and happens in a long term we can turn on sensors in steps of time to avoid aging in sensor's components (Delay element, FF, and XOR). This can be easily done by disconnecting the clock source (SCLK) from them (e.g., by setting a flag). Next, we present a comprehensive comparison between our proposed sensor and existing sensors presented in [5] and [6].

4 Sensible versus Previous Works

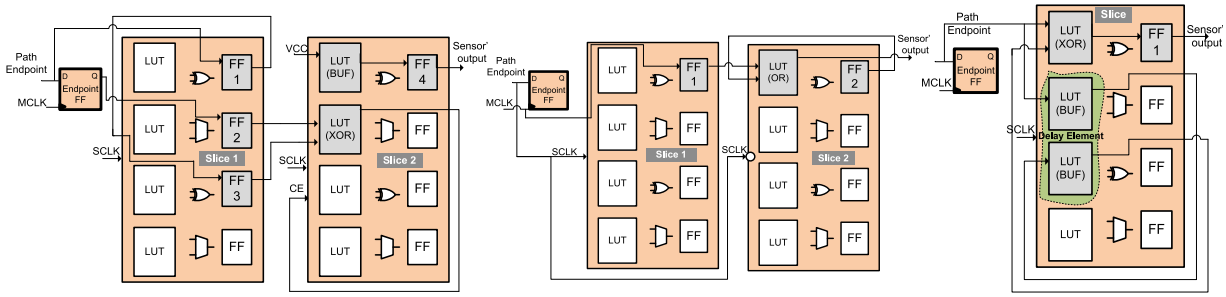
The proposed sensor in [5] as shown in Figs. 4a and 5a utilizes the observation (guard-band) interval (T_g) of the clock period for detecting any unwanted late transition due to aging. SCLK is the negative-shifted MCLK by T_g to store the correct data before late transition. Then, the faulty data after late transition will be stored in endpoint flip-flop for comparison. By using a T_g shift of MCLK for generation of SCLK, it only detects aging on the targeted RCP accurately (with same T_g).



a) Proposed sensor in [5] b) Proposed sensor in [6] c) SENSIBLE

Fig. 4. Sensor design.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.



a) Proposed sensor in [5] b) Proposed sensor in [6] c) SENSIBLE

Fig. 5. Sensor implementation on FPGA resources.

Hence, for multi-sensor insertion on different RCPs multiple clock generators are required because they have different T_g. We may be able to use shared SCLK as we proposed in this paper for this sensor but in cost of losing sensor sensitivity and accuracy significantly due to its described way of detecting timing violation. Furthermore, to avoid sensor sensitivity-loss FF1 and endpoint FF must be placed in the same CLB to avoid programmable inter-CLB routing resources with higher delay overhead. Clock enable (CE) input of FF4 is different from other FFs inside the sensor. Hence, we need at least two *slices* (sometimes three) to implement this sensor since flip-flops inside a slice have the same CE inputs (only one CE).

As shown in Figs. 4b and 5b, the sensor proposed in [6] uses the signal on the main CP's output as its SCLK and uses MCLK as its input. When critical path has late rising (falling) transition due to aging, the positive level of MCLK will be latched in FF1 (FF2) as the sign of aging. By using MCLK as sensor's input, it only detects aging on the main critical path. Hence, for multi-sensor insertion on different RCPs (with different delays), we are not able to share SCLK between different sensors and they need multiple clock generators for different detection windows. In addition, FF1 is a positive edge-triggered flip-flop and FF2 is negative, this means we need two *slices* to implement this sensor because flip-flops inside a *slice* only can be triggered either on the positive or negative edge.

In summary, the aforementioned sensors have two main drawbacks. For multi-sensor insertion, they need multiple clock generators which impact the sensor design scalability because of limited number of clock generators on FPGAs and increasing clock routing complexity. This becomes worse in multi clock domain designs in FPGAs. Designing and tweaking different SCLKs for these sensors will be a challenge for designers as well. Since these sensors cannot be placed at the same path endpoint CLB, the inter-CLB routing resources are required to connect the sensor to the path endpoint. This delay overhead impacts the sensitivity of such sensors or may even introduce new critical path to the circuit. Furthermore, using two slices (one CLB) for a sensor induces area and power overheads, which again impacts their scalability for large circuits.

5 Representative Critical Paths Selection

Due to limited number of resources in reconfigurable architectures, we are not able to monitor the large number of critical paths for aging mitigation techniques. Therefore, we use the amended version of our proposed algorithm in [8] to find the minimum number of RCPs. Algorithm 1 shows our filtering based RCP selection pseudo code. The algorithm inputs are the activity matrix $\vec{\alpha}$, duty cycle matrix $\vec{Y}$, power consumption matrix $\vec{P}$, clock frequency f, and matrix of each node (i.e., slice) Fan-out along each critical path $\overrightarrow{FO}$.

At the beginning the list of RCP (*RCPlist*) is equal to list of CPs. For each CP, we calculate the delay, stress, and fan-out using their node level information (matrices) (lines 2-6). Temperature is the dominant factor for BTI and HCI aging mechanisms. Since each CP goes across different regions of implemented design on FPGA the nodes (i.e., slice) along it will experience different temperatures and stresses (Appendix A, Fig. 10). Hence, we call HotSpot [15] to extract temperature map at node level as well. Then we remove CPs from the RCPlist that experiences lower temperature than the temperature threshold (lines 7-15). After that, we extract the stress map using activity and duty cycle matrices at same level. Then CPs from *RCPlist* that suffer less than the stress threshold are removed (lines 16-20).

Algorithm 1. RCP Selection

Input: Critical paths list {CP}, Delay matrix $\vec{d}$, Activity matrix $\vec{\alpha}$, Duty cycle matrix $\vec{Y}$, Power consumption matrix $\vec{P}$, Clock frequency f, Fan-out matrix $\overrightarrow{FO}$,Path endpoints list {*Ph*}
Output: List of representative critical paths {RCP}

1. $RCPlist = \{CP\}$;
2. **for** each $Path_i \in \{CP\}$ **do**
3. $D_i \leftarrow CalDelay\ (Path_i,\ \vec{d})$;
4. $Stress_i \leftarrow CalStress\ (Path_i,\ \alpha_i, f,\ Y_i)$;
5. $FO_i \leftarrow CalFO(Path_i,\ \overrightarrow{FO})$;
6. **End for**
7. $\vec{T} \leftarrow FindTemp\ (\vec{P})$; // Call HotSpot
8. **for** each $Path_i$ **do**
9. $Temp_i \leftarrow CalAvgTemp(Path_i, \vec{T})$;
10. **End for**
11. $T_{th} \leftarrow CalTempThreshold()$;
12. **for** each $Path_i \in RCPlist$ **do**
13. **if** $Temp_i < T_{th}$ **then**
14. $RCPlist.Remove(Path_i)$;
15. **End for**
16. $Stress_{th} \leftarrow CalStressThreshold()$;
17. **for** each $Path_i \in RCPlist$ **do**
18. **if** $Stress_i < stress_{th}$ **then**
19. $RCPlist.Remove(Path_i)$;
20. **End for**
21. $delay_{range} \leftarrow CalDelayRange()$;
22. **for** each $Path_i \in \{RCPlist\}$ **do**
23. **if** $Path_{i-delay} \notin delay_{range}$ **then**
24. **if** $CalAging(Path_i) <$ aging-rate$_{\max}$ **then**
25. $RCPlist.Remove(Path_i)$;
26. **End for**
27. $FO_{th} \leftarrow CalFOThreshold()$;
28. **for** each $Path_i \in RCPlist$ **do**
29. **if** $FO_i < FO_{th}$ **then**
30. $RCPlist.Remove(Path_i)$;
31. **End for**
32. $RCPlist.Remove\ (Ph)$;
33. **Return** RCPlist;

At this step, the *RCPlist* contains CPs that age faster than removed CPs. To reduce the performance overheaddue to sensor

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.

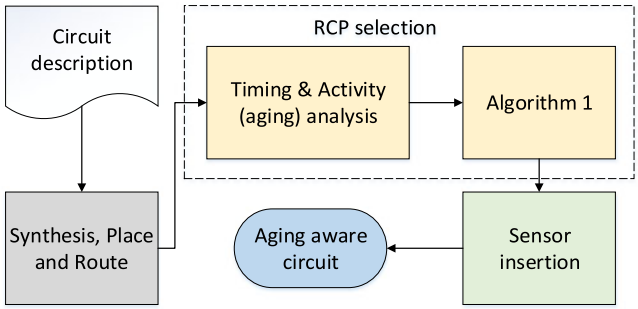


Fig. 6. Sensor insertion flow.

insertion on RCPs, it is better to remove CPs with smaller *slack times* and lower aging rates. Therefore, we find RCPs that have higher aging rates than CPs with smaller *slack times* in comparison with the main CP of the circuit. By monitoring such paths, we can detect aging sooner in order to react accordingly. This is shown in our previous work [8] that selected RCPs may have higher aging rates than CPs with the higher delay (i.e., smaller slack time). Hence, we define a delay range ($delay_{range}$) for RCPs selection. The delay upper bound (β) is computed by the negation of critical path delay by the sensor delay and the lower bound (γ) delay is defined by the user (e.g., 90 percent of critical path delay). To avoid timing failure of CPs beyond upper bound (β) (including main CP), we consider the aging rates. If their aging rates are slower than selected RCPs, then it is sufficient (guarantee the timing failure will not happen, because aging will be captured sooner in the already selected RCPs), otherwise that path will be in the *RCPlist* (lines 21-26).

To fairly distribute the sensors among the chip and to reduce the number of RCPs we consider their endpoint physical location (Ph) and average number of fanouts (FO) along a path's nodes (i.e., slices) (lines 27-32). The paths with higher average fanout will have higher impact on the reliability of implemented circuit (aging propagation probability is higher). More detailed explanation of Algorithm 1 and RCP selection method is in our previous work [8]. In this work, we improved the algorithm by changing the filtering steps in order to avoid removing CPs with high aging rates. As mentioned earlier, we first keep CPs with higher aging rates (lines 11-20) then remove those ones from the list that do not satisfy the delay range threshold (lines 21-26). By online monitoring RCPs we can detect aging to react accordingly (change the configuration bits) before timing failure happens. For instance, related works in [11], [13] can use sensors aging monitoring.

6 Experiments

6.1 Setup

Experiments using Aritx7-based FPGA board have been performed in order to evaluate, validate, and analyze SENSIBLE. The implementation of the proposed sensor is feasible with different versions of Xilinx tools (ISE) but can be adapted easily to other vendor or third-party tools. Three benchmarks (DCT (large), AES (medium) and FIR (small)) have been used in our experiments to evaluate the impact of SENSIBLE on area, power, and performance, while assessing the results in term of accuracy and sensitivity. Using Algorithm 1 we find maximum number of required RCPs (sensors), which are 35, 21, 17 sensors (RCPs) for AES, DCT and FIR, respectively. In our experiments, different numbers of sensors (5, 10, and 15) are placed in the top selected RCPs of the circuits with the highest aging rate using our placement tool that uses Xilinx Design Language (XDL). To extract power consumption and performance overhead Xpower Analyzer and TRACE by Xilinx are utilized, respectively.

TABLE 1
SENSIBLE Overheads for Different Numbers of Inserted Sensors

Bench. \ #sensor	Power (%)			Area (%)			Perf. (%)
	5	10	15	5	10	15	5, 10,15
AES	0.61%	0.53%	0.65%	0.30%	0.70%	1.20%	0.30%
FIR	0.83%	1.10%	1.24%	1.50%	3.00%	4.60%	0.00%
DCT	1.26%	1.46%	1.61%	1.10%	2.30%	3.10%	0.00%
AVG.	**0.90%**	**1.03%**	**1.17%**	**0.97%**	**2.00%**	**2.96%**	**0.10%**

6.2 Sensor Insertion

Our automated sensor insertion flow is shown in Fig. 6. First, a synthesis tool (Xilinx ISE) gets the circuit description in a standard HDL format. Design parameters including path delays (by timing analyzer), wire and node activities for aging estimation (by Xilinx Power Analyzer), and nodes locations (by Xilinx PlanAhead) is extracted after the place and route step. Now, paths are ranked based on their aging rates extracted by these parameters. Using these analyses, RCPs selection will be done by Algorithm 1.

A greedy local search algorithm finds unused resources in the closest slice (for our sensor) or closest CLB near the endpoint slices of selected RCPs. Hence, *Data Flow Graph* (DFG) of the placed and routed design is extracted from the XDL file. Each node of the DFG is a logic block (i.e., slice) that retains information about used and unused resources inside it (e.g., LUT, FF, and etc.). Furthermore, each logic block (or node) is distinguished by physical placement on the FPGA through its x- and y-coordinates. Using this information, we can find the used or unused resources in neighbor logic blocks. Our greedy local search algorithm looks for the first found closest unused resources for placing the sensors required resources (e.g., FF and LUT) [16].

After placement of the sensor, we use the router of commercial tool (ISE) to route our sensors' resources. Before using the router, our tool routes the placed resource partially, meaning it determines which inputs or outputs will be connected. This new DFG, after the placement and routing of sensors (i.e., insertion), will be converted back to the XDL format for generating the bitstream of the FPGA based design. All the insertion steps are done on the XDL file, which is compatible with commercial tools. Therefore, our sensor insertion is independent of the implemented FPGA design and can be automated and embedded to ISE or any third-party tools.

As shown in Fig. 6, sensor insertion will be done after original design implementation by commercial vendor tools which consider the worst case (considering process variation) in their placement and routing algorithms. In other word, the sensor insertion flow is in-place and tries not to change the optimized placement and routing by using unused resources after placement and routing. Additionally, FPGA vendors do not release their device level information in order to extract process variation details at the logic level. There are few techniques to extract process variation information of FPGAs using *Ring Oscillator* (RO) [17], [18]. Using these techniques, we can extract process variation information and consider it in our RCP selection algorithm.

6.3 Results and Discussion

Table 1 shows the area (total number of slices), power (total dynamic and static power), and performance overheads (sensor insertion on RCPs may introduce new critical path, thus we will have performance overhead) for the proposed sensor architecture and design. For fair comparison, area and speed are chosen as optimization goals in the synthesis phase for area and performance overhead calculation, respectively. As shown in Table 1, the average power overhead of AES, FIR, and DCT benchmarks for 5, 10, and 15 sensors is 0.90, 1.03, and 1.17 percent, respectively. This low power overhead is not only due to the fact that the proposed sensor only occupies one slice but also regardless of number of sensors,

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
AVG. Overheads Comparison (for 15 Inserted Sensors)

Arch.	Power	Area		Perf.
		#slice	#MMCM	
SENSIBLE	1.17%	2.96%	1	0.10%
[5]	3.10%	7.20%	2	1.14%
[6]	4.43%	6.13%	2	2.62%

we only use one CMT (MMCM) for generating SCLK as well as MCLK (each MMCM can generate 7 different clock frequencies at the same time).

When ten sensors are inserted for monitoring aging in AES, FIR, and DCT, the area overhead is 0.70, 3.00, and 2.30 percent, respectively. Our aging sensors are inserted in selected RCPs' endpoints with high aging rates and are designed using resources inside one slice only. Hence, they are most likely placed in the same endpoint CLB (using available unused slice). The performance overhead of sensor insertion is negligible (i.e., it does not introduce longer critical path to the circuit).

In Table 2, the average area, power, and performance overheads using SENSIBLE and two other implemented aging sensors [5], [6] are reported for selected benchmarks when 15 sensors are inserted. Results show that our sensor has lower overhead compared to the two other sensors. This is because that our sensor regardless of number of required sensors only uses one CMT (MMCM) for generation of MCLK and SCLK (shared), while other proposed sensors [5], [6], as shown in Table 2, require 2 MMCMs for 15 sensors. This number increases by increment in the number of sensors (different SCLKs are required). Additionally, our sensor only occupies one slice while other sensors require two slices (a CLB). This also leads to lower overheads in terms of area, power and performance. If we use larger designs in comparison to the mentioned benchmarks, they may need higher number of sensors. Additionally, if we decide to monitor higher number of CPs, we need higher number of sensors as well. Since SENSIBLE overhead is minimum (one slice and one clock generator), it outperforms previous aging sensor design in every aspect of above-mentioned overheads.

In order to measure and compare the sensitivity (accuracy) of SENSIBLE with prior works, we implemented the circuits and inserted the aging sensors in the implemented benchmarks on Artix7 FPGA and connected the sensors' flag to LEDs on the board. As shown in the flow in Fig. 7, to emulate aging impact on the implemented benchmarks on FPGA, we increase the delay on RCPs by small steps of Δd. Next, we translate the delay degradation ($\Delta d = d_{aged} - d_0$) to frequency and increase clock frequency (MCLK) by $\Delta f = \frac{\Delta d}{d_0 \times d_{aged}}$. This occurs iteratively until the connected sensor outputs to the LED are set and the LED turns on. By this means, we can find the amount of (Δd), or frequencies, at which the implemented sensors will be triggered. Using HotSpot [15] for temperature simulation, extracting the stress rate (duty cycle and activity) after running benchmarks, and delay degradation Δd resulting from on-board experiment, we calculate the earliest time the sensors detect aging-induced delay degradation of Δd using Eqs. (1) and (2) in Appendix A. Please consider that we emulated aging by increasing clock frequency of the implemented design to find the "relative" (as opposed to exact) amount of improvement in accuracy of SENSIBLE in comparison to previous works.

Fig. 8 demonstrate the aging along top selected RCP in each application. The highlighted points on the curve shows the earliest time and the amount of delay degradation detected by three different aging sensors from the flow in Fig. 7. For example, in AES, our proposed aging sensors will be triggered after 9.78 weeks when the induced delay is 122ps (point (9.78, 0.122)). The proposed sensors in [5] detect aging after almost 33 weeks when the degraded delay is 193ps. These values for sensors in [6] are ~38 weeks and 238ps, respectively. The results show that our sensor is more sensitive to aging induced delay degradation and therefore, is more accurate.

Fig. 9 summarizes the distance between aging sensors and the endpoint flip-flop of the paths under monitoring. While in our flow, we provide stringent constraints to design tools to place the aging sensors in the same CLB as endpoint flip-flop (distance of zero), the tool will apply best effort but cannot guarantee and hence, in some cases, places the sensors in nearby CLBs (so called sensor displacement). The results show that we can decrease this displacement to zero by forcing RCP endpoint CLB to leave a slice

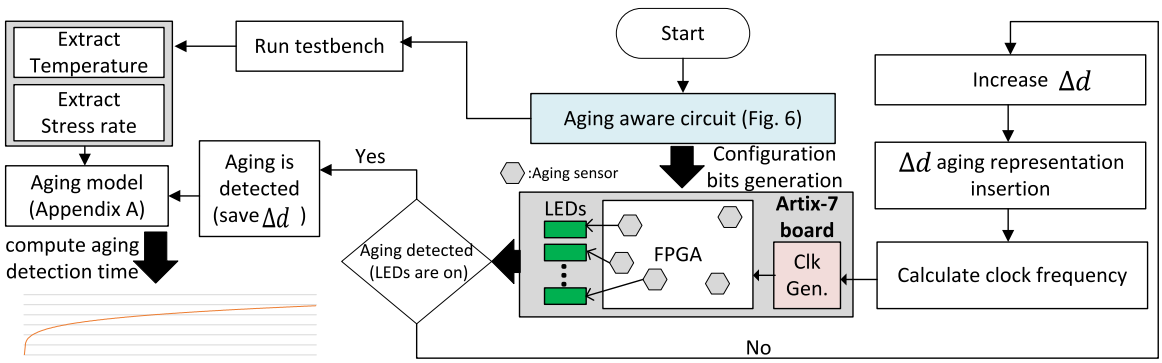


Fig. 7. On board sensor sensitivity-test flow.

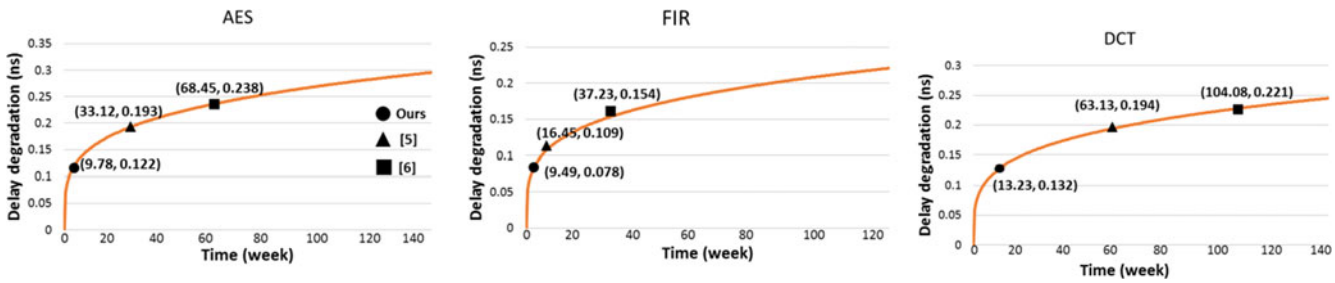
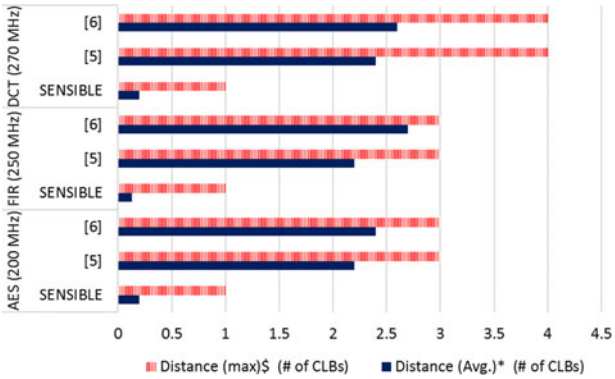


Fig. 8. Aging sensitivity comparison.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.



*Average distance of sensor's CLB from path's endpoint CLB

$ Worst case distance of sensor's CLB from path's endpoint CLB

Fig. 9. Aging sensor displacement (15 inserted sensors).

unused. We are not able to do this for sensors in [5], [6] since they need at least a CLB due to their multiple clock design that cannot be supported using one slice. In the worst case, our proposed sensor is placed in the adjacent CLB (max distance = 1 CLB).

Results show that aging sensor displacement in our design is significantly lower than displacement for aging sensors in [5], [6]. The aging displacement results correlate with the sensor sensitivity results as shown in Fig. 9. The higher is the average distance, the longer is the detection time. Since prior works need at least two slices for their implementation, this forces to use inter-CLB routing (switch-boxes) resources to connect them to selected RCP's endpoint which impacts their sensitivity (accuracy). It should be noted that aging induced delay degradation is very small delay that can be easily compensated by routing delay in aging sensors.

Earlier aging detection of SENSIBLE in comparison to previous works results in better balance of aging among resources on an FPGA and decreases the guardband of critical path. As a result, earlier aging detection can be a better guidance to system level task dispatching on implemented designs' block in FPGAs. Furthermore, placement and routing tools (e.g., ISE) can allocate smaller guardband which increases the implemented designs' performance drastically. In all, higher sensitivity results in faster online reaction in system level aging mitigation techniques and higher system reliability.

As discussed in Algorithm 1, aging sensors will be inserted in different regions of the FPGAs and the clock delay (clock skew) may occur to sensors. While, for our experiments on the Artix-7 board we have not encountered such a problem, there are few techniques in the literature to avoid this issue (so called clock deskew) [19], [20]. Furthermore, for the state-of-the-art FPGAs clock network deskew is provided [21].

7 Conclusion

In this paper, SENSIBLE, the architecture and design of a low-overhead aging (timing) sensor is proposed along with the strategy for its clock design to increase its scalability in multi-sensor applications. Then, we discussed and compared SENSIBLE with two available aging sensors for FPGAs. We implemented these sensors in selected benchmarks on Artix7 FPGA board for real world comparisons. SENSIBLE exceeds prior works by lower overheads and earlier detection of aging in designs (higher accuracy). As future work, we plan to use our proposed sensor design for online aging mitigation of application on FPGAs such as datacenters.

Appendix A
Aging-Induced Delay Degradation Model

Delay degradation at transistor level due to BTI and HCI mechanisms results in delay degradation in logic resources of FPGAs

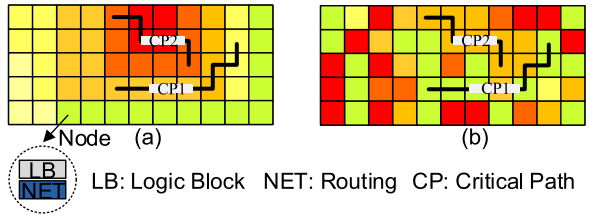


Fig. 10 . Temperature (a) and stress (b) maps impact on critical path aging.

such as LUTs, FFs, and carry chains [4], [7], [22]. BTI is a static mechanism and occurs when temperature (T) is high and transistor is constantly under stress (ON). BTI-based degradation depends on transistor's duty cycle (Y) and includes two phases. When the transistor is ON, it is in its stress phase, hence, increasing the threshold voltage (Vth) of transistor. This manifests itself as delay degradation on the FPGA resources (i.e., FF, LUT, MUX, routing resources). When transistor is OFF, it is in its recovery phase and Vth will partly decrease. This process manifests itself as partial delay recovery of FPGA resources. The delay degradation due to BTI at time t is a function of duty cycle (Y) and temperature (T) for intrinsic delay (d_0) [32], [34]:

$$\Delta d_{BTI}\ (t) = A_{BTI}\ \times (Y)^n \times (t)^n \times e^{\left(\frac{-E_a}{kT}\right)} \times d_0 \qquad (1a)$$

where, A_{BTI} is technology dependent factor, n is a constant depending on fabrication process, k is Boltzmann's constant, E_a is activation energy.

HCI is a dynamic mechanism that occurs when temperature is high and transistor is toggling. HCI-based degradation depends on transistor switching activity. It changes the current-voltage characteristic of transistor that increases Vth. The delay degradation due to HCI at time t is a function of clock frequency (f), activity (α), and temperature (T) [22], [24]:

$$\Delta d_{HCI}\ (t) = A_{HCI}\ \times \alpha \times f \times (t)^{0.5} \times e^{\left(\frac{-E_b}{kT}\right)} \times d_0 \qquad (2a)$$

where, A_{HCI} is technology dependent factor, k is Boltzmann's constant, E_b is activation energy. BTI aging effect on delay degradation is more dominant than HCI aging effect [8], [22]. Duty cycle (Y) and clock frequency multiplied by activity ($\alpha \times f$) are known as stress in BTI and HCI, respectively.

Since the device level information of the FPGA is not disclosed by vendors, we deployed the proposed method in [8], [22] to calculate aging of nodes along a CP. Node is a basic logic element and its corresponding routing resources (Slice and NET in Xilinx Artix). We assume that all transistors inside a node experience similar stress and temperature. Delay degradation along each path is dependent on the activity and temperature of the resources along the path. As shown in Fig. 10, due to different temperatures and stress rates (duty cycle, activity and clock frequency) of resources on FPGA, RCPs experience different aging rates [8]. It is observed that such paths may experience higher aging rates than critical paths and their delay may exceed critical paths' due to aging.

References

[1] S. Borkar, "Designing reliable systems from unreliable components: the challenges of transistor variability and degradation," *IEEE Micro* vol. 25, no. 6, pp. 10–16, Nov./Dec. 2005.
[2] K. Bernstein, et al., "High-performance CMOS variability in the 65-nm regime and beyond," *IBM J. Res. Develop.*, vol. 50, no. 4.5, pp. 433–449, 2006.
[3] G. Yan, F. Sun, H. Li, and X. Li, "CoreRank: Redeeming imperfect silicon by dynamically quantifying core-level healthy condition of manycore processors," *IEEE Trans. Comput.*, vol. 65, no. 3, pp. 716–729, Mar. 2016.
[4] E. A. Stott, J. S. J. Wong, P. Sedcole, and P. Y. K. Cheung, "Degradation in FPGAs: Measurement and modelling," in *Proc. Field Programmable Gate Arrays*, 2010, pp. 229–238.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.

[5] M. D. Valdes-Pena, et al., "Design and validation of configurable online aging sensors in nanometer-scale FPGAs," *IEEE Trans. Nanotechnology*, vol. 12, no. 4, pp. 508–517, Jul. 2013.
[6] A. Amouri and M. Tahoori, "A low-cost sensor for aging and late transitions detection in modern FPGAs," in *Proc. Field Programmable Logic Appl.* 2011, pp. 329–335.
[7] S. Srinivasan, et al., "Toward increasing FPGA lifetime," *IEEE Trans. Dependable Secure Comput.*, vol. 5, no. 2, pp. 115–127, Apr.-Jun. 2008.
[8] M. Ebrahimi, et al., "Path selection and sensor insertion flow for age monitoring in FPGAs," in *Proc. Des. Automat. Test Europe*, 2016, pp. 792–797.
[9] M. Omana, et al., "Low cost NBTI degradation detection and masking approaches," *IEEE Trans. Comput.*, vol. 62, no. 3, pp. 496–509, Mar. 2013.
[10] G. Yan, Y. Han, and X. Li, "ReviveNet: A self-adaptive architecture for improving lifetime reliability via localized timing adaptation," *IEEE Trans. Comput.*, vol. 60, no. 9, pp. 1219–1232, Sep. 2011.
[11] Z. Ghaderi, and E. Bozorgzadeh, "Aging-aware high-level physical planning for reconfigurable systems," in *Proc. Asia South Pacific Des. Automat. Conf.*, 2016, pp. 631–636.
[12] M. Omaña, et al., "Novel low-cost aging sensor," in *Proc. Int. Conf. Comput. Frontiers*, 2010, pp. 93–94.
[13] K. M. Zick and J. P. Hayes, "On-line sensing for healthier FPGA systems," in *Proc. Field Programmable Gate Arrays*, 2010, pp. 239–248.
[14] A. A. M. Bsoul, et al., "Reliability-and process variation-aware placement for FPGAs," in *Proc. Conf. Des. Automat. Test Europe*, 2010, pp. 1809–1814.
[15] K. Skadron, et al., "Temperature-aware microarchitecture," *ACM Comput. Archit. News*, vol. 31, no. 2, pp. 2–13, 2003.
[16] Z. Ghaderi, S. G. Miremadi, H. Asadi, and M. Fazeli, "HAFTA: Highly available fault-tolerant architecture to protect SRAM-based reconfigurable devices against multiple bit upsets," *IEEE Trans. Device Materials Rel.*, vol. 13, no. 1, pp. 203–212, Mar. 2013
[17] H. Yu, X. Qiang, and P. H. Leong, "Fine-grained characterization of process variation in FPGAs," in *Proc. Field-Programmable Technol.*, 2010, pp. 138–145.
[18] A. Bsoul, N. Manjikian, and L. Shang, "Reliability-and process variation-aware placement for FPGAs," in *Proc. Conf. Des. Autom. Test Europe*, 2010, pp. 1809–1814.
[19] C. Dike, N. A. Kurd, P. Patra, and J. Barkatullah, "A design for digital, dynamic clock deskew," in *Proc. Int. Symp. VLSI Circuits*, 2003, pp. 21–24.
[20] K. Fujiwara, K. Kawamura, M. Yanagisawa, and N. Togawa, "A high-level synthesis algorithm for FPGA designs optimizing critical path with interconnection-delay and clock-skew consideration," in *Proc. Symp. VLSI Des. Automat. Test*, 2016, pp. 1–4.
[21] Xilinx, 7 Series FPGAs Clocking Resources, 2016.
[22] A. Amouri and M. Tahoori, "High-level aging estimation for FPGA-mapped designs," in *Proc. Field Programmable Logic Appl.*, 2012, pp. 284–291.
[23] M. Noda, et al., "On estimation of NBTI-induced delay degradation," in *Proc. Test Symp. Eur.*, 2010, pp. 107–111.
[24] A. Bravaix, et al., "Hot-carrier acceleration factors for low power management in DC-AC stressed 40nm NMOS node at high temperature," in *Proc. Rel. Phys. Symp.*, 2009, pp. 531–548.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO CEARA. Downloaded on April 19,2025 at 03:13:02 UTC from IEEE Xplore. Restrictions apply.